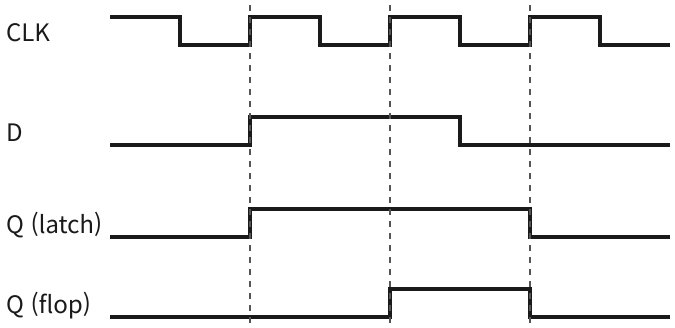
L12. Timing Methodologies and Registers

Lecture 12

12.1 Latch 와 flip-flop 복습

> **Latch = level sensitive, flip-flop = edge triggered.**

	Latch	Flip-flop
동작	level sensitive (transparent). clock(En)이 high 인 동안 출력이 D 를 따르고 low 이면 값을 유지	edge triggered. clock edge 에서만 출력이 바뀐다 (opposite clock 의 master + slave latch)
다른 이름	transparent latch, D latch	master-slave flip-flop(구조 방식), D flip-flop, D register
특징		master + slave latch 두 개 구조라 latch 보다 면적이 큼. 대신 edge 에서만 입력을 샘플링하므로 edge 사이의 입력 변화는 Q 에 닿지 않음



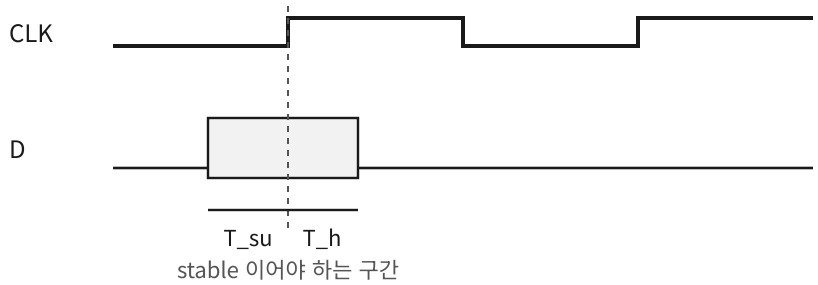
Latch 는 clock 이 high 인 동안 D 를 따라가고(투명), flip-flop 은 rising edge 의 값만 잡아 한 주기 동안 유지한다. (모양은 개념도)

12.2 Timing methodology

Flip-flop 의 타이밍 제약

- Setup time T_{su}: clock edge 전에 data 가 안정(변화 없음)해야 하는 시간.

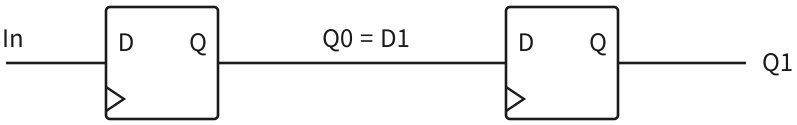
- Hold time T_h: clock edge 후에 data 가 안정해야 하는 시간.
- clock → Q delay = clock edge 부터 출력 Q 가 새 값으로 전파되기까지의 지연. 이 delay 가 지나야 F/F 출력이 안정됨. setup/hold 는 입력 D 쪽 제약이고 이것은 출력 쪽 지연.



Setup time T_su 는 clock edge 이전, hold time T_h 는 clock edge 이후에 data 가 안정해야 하는 시간이다.

연결된 flip-flop (shift register)

- D flip-flop 두 개를 이어 2단 shift register 구성. 입력 In = "0101011" 이 첫 비트부터 clock 마다 한 단씩 전파됨. T_p = propagation delay.
- data 는 clock 한 주기에 F/F 하나씩 통과. 만족해야 할 타이밍 제약은 setup 과 hold 두 가지.



CL 이 있으면 두 F/F 사이에 조합 논리가 들어간다

연결된 두 flip-flop (2단 shift register). 같은 clock 이 둘 모두에 들어간다.

Setup 과 propagation

- T_p: 두 F/F 사이 경로의 전체 delay. 기본은 F/F0 의 clock → Q0 (= D1) delay. T_{period}: clock 주기. T_{su}: setup time.

Timing constraint 1

$T_p < T_{period} - T_{su}$

$T_{period} > \text{Delay(F/F0)} + T_{su}\text{(F/F1)}$

- 사이에 조합 논리(CL)가 있으면 T_p = (F/F0 의 clock → Q0 delay) + (Q0 → D1 조합 논리 delay). 같은 제약 $T_p < T_{period} - T_{su}$ 에 이 값을 넣음.

Hold 와 propagation

- Hold time violation: F/F0 의 출력이 너무 일찍 바뀌어 F/F1 의 hold time 이 끝나기 전에 D1 이 바뀜. D1 은 hold time 이 끝난 뒤에 바뀌어야 함.

> **Timing constraint 2**
>
> $T_p > T_h$ (hold time violation 회피)

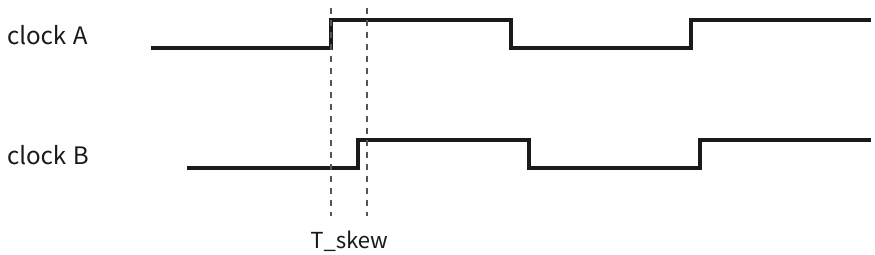
Positive edge triggered 시스템 정리

> Setup: $T_{p,max} < T_{period} - T_{su}$
>
> Hold: $T_{p,min} > T_h$

- setup 은 가장 느린 경로, hold 는 가장 빠른 경로의 delay 로 계산.
- hold violation 은 clock 주파수를 낮춰도 안 없어짐. hold 제약에 T_{period} 가 없기 때문. 해결은 data 경로에 buffer 넣기, 또는 clock skew 줄이기.

Clock skew

- Clock skew: 같은 source 의 clock 이 위치에 따라 다른 시각에 도착하는 현상. clock B 가 늦으면 skew 는 양수.



Clock skew: clock B 가 늦게 도착하면 skew 는 양수다 (T_skew = t(clock B) − t(clock A)).

skew 를 넣은 제약

Setup: $T_p < T_{period} - T_{su} + T_{skew}$

Hold: $T_p > T_{skew} + T_h$

Setup / Hold / skew 계산기 (예제 값: T_cq 2, CL 5/1, T_su 1, T_h 1)

Tp,max = Tcq + CLmax = 7.00 ns, Tp,min = Tcq + CLmin = 3.00 ns

Setup: Tp,max ≤ Tperiod − Tsu + Tskew → 여유 0.00 ns OK (최소 주기 8.00 ns, fmax ≈ 125 MHz)

Hold: Tp,min ≥ Tskew + Th → 여유 2.00 ns OK

Tskew = t(clock B) − t(clock A). 양수이면 setup에 유리하고 hold에 불리하다. 여유가 0이면 해당 제약은 등호로 만족한다.

- 양의 skew 는 setup 에 유리. 두 번째 F/F 의 clock edge 가 늦어져 data 가 안정될 시간이 늘어남.
- 반면 hold 에는 불리. 두 번째 F/F 의 clock edge 가 늦으니, 첫 F/F 의 새 data 가 hold time 이 끝나기 전에 도착해 버릴 수 있음.

예제: 타이밍 제약

문제

- 주어진 값: T_{cq} = 2 ns (두 F/F 모두 clock → Q delay).
- CL delay = 5 ns (max) / 1 ns (min).
- T_{su} = 1 ns, T_h = 1 ns.
- T_{skew} = t(clock B) − t(clock A) (양수이면 clock B 가 늦게 도착).

Q1. skew 가 없을 때 (T_{skew} = 0): (a) 최소 clock 주기 T_{period} 와 최대 주파수 f_{max}. (b) hold 제약을 만족하는가, 여유는?

Q2. clock B 가 1.5 ns 늦게 도착 (T_{skew} = +1.5 ns): (a) 최소 주기. (b) hold 여유.

Q3. T_{skew} = +2.5 ns: 어떤 주파수에서든 동작하나? 안 되면 어떻게 고치나?

풀이

경로 delay: $T_{p,max} = T_{cq} + CL_{max} = 2 + 5 = 7$ ns, $T_{p,min} = T_{cq} + CL_{min} = 2 + 1 = 3$ ns.

- Q1. Setup: $T_{p,max} + T_{su} < T_{period}$ 이므로 경계 주기 8 ns, 경계 주파수 125 MHz. setup 여유를 두려면 주기는 8 ns 보다 크게, 주파수는 125 MHz 보다 작게 잡음.
- Hold: $T_{p,min} > T_h$ 에서 3 ns > 1 ns 이므로 OK, 여유 2 ns (T_{period} 는 나오지 않는다).
- Q2. Setup: $T_{period} > 7 + 1 - 1.5 = 6.5$ ns, $f_{max} \approx 154$ MHz (양의 skew 가 setup 에 도움).
- Hold: $T_{p,min} > T_{skew} + T_h$ 에서 3 ns > 2.5 ns 이므로 OK, 여유는 0.5 ns 로 줄어든다.
- Q3. Hold: 3 ns > 3.5 ns ? 위반이다.
- 어떤 clock 주파수에서도 위반이다 (clock 을 느리게 해도 소용없다).
- 해결: data 경로에 0.5 ns 보다 큰 buffer 를 넣거나(정확히 0.5 ns 는 여유 0) skew 를 줄인다(clock tree balancing).
- 예: 1 ns buffer 를 넣으면 $T_{p,min} = 4$ ns > 3.5 ns 로 OK. buffer 는 최대 경로에도 더해지므로 setup 경계 주기도 8 + 1 − 2.5 = 6.5 ns 로 1 ns 늘어남.

> - 정리.
> - setup 은 가장 느린 경로 대 T_{period} (clock 을 늦추면 해결).
> - hold 는 가장 빠른 경로 대 $T_h + T_{skew}$ (clock 주기가 식에 없음).

Metastability 와 synchronization failure

- F/F 입력이 clock edge 근처(setup 과 hold 구간 안)에서 바뀌면 어떻게 되는가?
  - F/F 이 metastable 상태에 들어갈 수 있다.
  - Metastability: Q 가 한동안 유효한 logic 0/1 전압 범위에 도달하지 못하는 상태.
  - setup/hold 구간 안에서 D 가 바뀌면 Q 가 metastable 상태가 될 수 있음.

Asynchronous input

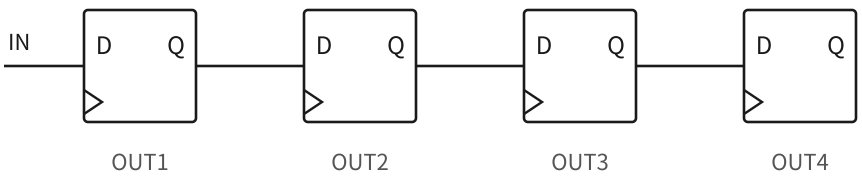
- 비동기 입력은 F/F 입력에 아무 때나 도착해서 synchronization failure 를 일으킬 수 있음.
- 처리: synchronizer. 비동기 입력을 쓰기 전에 시스템 clock 으로 구동하는 F/F 로 샘플링. F/F 두 개를 직렬로 쓰는 게 일반적: 두 번째 F/F 가 첫 F/F 출력이 안정될 시간을 벌어 metastability 가 후속 회로로 전달될 확률을 줄임. 실패 확률은 줄지만 0 은 못 만듦.

12.3 Register (교재 6.3)

- Register: 여러 F/F 를 묶어 하나의 다중 비트 값을 저장하는 회로. 각 F/F 가 한 비트를 저장하고, clock, reset, set 선을 공유해 관련된 비트를 함께 갱신.
- 예: parallel register, shift register. 4입력 register 는 F/F 4개가 IN1~IN4 를 병렬로 받아 OUT1~OUT4 를 냄.

Shift register

- 4-bit shift register: 매 clock 마다 새 입력이 첫 F/F 에 저장되고 기존 값은 다음 F/F 로 이동. 최근 입력 4개를 순서대로 보관. 병렬 출력은 OUT1~OUT4, 직렬 출력은 OUT4. "Quad right-shifting shift register" 는 직렬 입력을 받는다.



4-bit shift register: D flip-flop 4개를 직렬로 연결하고 같은 clock 을 쓴다. 최근 입력 4개를 순서대로 보관.

Serial-parallel 변환

- shift register 는 직렬 data 와 병렬 data 를 서로 변환. Serial-in parallel-out (SIPO): 직렬로 들어와 병렬로 나감. Parallel-in serial-out (PISO): 병렬로 로드해 직렬로 내보냄.

Universal shift register

- 한 단(N번째 cell)을 보면 다음 clock 의 새 값은 control 입력 s0, s1 과 clear 로 정해진다. 각 cell 앞에 mux 가 있고 입력은 Q[N−1] (왼쪽 cell), Q[N+1] (오른쪽 cell), Input[N] 과 자기 자신이다.

clear	s0	s1	new value
1	-	-	0
0	0	0	output (hold)
0	0	1	left FF 의 출력 (shift right)
0	1	0	right FF 의 출력 (shift left)
0	1	1	input (load)